

Sample-Efficient Sim-to-Sim Transfer via Curriculum Mass Randomization and Entropy Control

Ali Vaezi*, Yousef Fayyazi[†], Sajjad Shahali Ramsheh[‡], Parastoo Hashemi Alvar[§]

Abstract—Bridging the sim-to-real gap in reinforcement learning (RL) requires policies that remain reliable under unknown physical dynamics. We investigate this problem in MuJoCo HOPPER by imposing a 30 % torso-mass mismatch between the training and evaluation domains. Building on Proximal Policy Optimization (PPO), we couple *Curriculum Domain Randomization*, which gradually widens mass ranges, with a simple *Entropy Scheduling* strategy that anneals exploration. Classical policy-gradient baselines confirm the gap: REINFORCE variants learn quickly but stagnate, whereas Actor-Critic improves slowly yet steadily. The proposed PPO + CDR + ES agent surpasses all six ablated PPO variants in both sample efficiency and robustness: it crosses the 5 k-return mark in roughly 3.2×10^5 environment steps and maintains the highest area-under-curve across mass perturbations. Averaged over three seeds, it improves cumulative return by 72 % relative to vanilla PPO and more than quadruples that of PPO with uniform randomization. These findings show that a lightweight curriculum, paired with principled entropy decay, yields markedly more transferable and data-efficient locomotion policies without additional hyper-parameter tuning.

Index Terms—Sim-to-Real Transfer, Reinforcement Learning, Domain Randomization, PPO, Curriculum Learning, Entropy Scheduling

I. INTRODUCTION

Reinforcement Learning (RL) has become a key method for teaching robots how to perform tasks by trial and error in simulated environments [1]. With time, RL systems have improved from basic tasks to more complex movements, using tools like MuJoCo and PyBullet [2]. However, transferring these learned skills from simulation to the real world is still difficult. This problem is called the "sim-to-real gap" and is caused by differences between simulated and real environments [3]. These differences may include missing physical effects, hardware delays, or inaccurate models of contact and friction [4]. Solving this gap is important for making robots more reliable and usable in real-world settings [5]. A useful starting point is therefore a *zero-shot* evaluation: train in the source domain, freeze the weights, and test once in the target domain with no further gradient steps. The performance

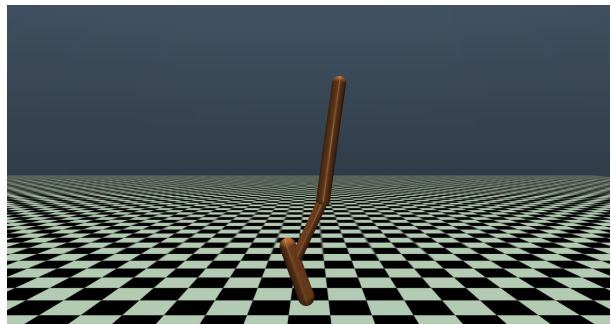


Fig. 1: MuJoCo HOPPER used as source domain. The torso mass is reduced by 30 % during training to create a controlled dynamics gap.

drop in this setting quantifies the raw difficulty of the mass-mismatch [6].

A common way to make RL policies more robust is by using *domain randomization* (DR). In DR, the simulator randomly changes environment dynamics like friction, motor strength, or object weight during training [7]. This helps the agent learn behaviors that can still work even when the environment changes slightly [8]. DR turns the simulator into a dynamic training ground with many possible variations, allowing agents to generalize better to new situations [9]. Although DR was first used for visual tasks, it has recently shown promises in physical control tasks as well [10].

Static Domain Randomization (SDR) is a version of DR where the changes follow fixed random ranges during the entire training process [11]. Studies have shown that even simple changes in mass or sensor noise can help agents succeed in new environments [12]. SDR is simple and effective, but it depends heavily on how the random values are chosen [13]. However, if the randomization range is too narrow, the agent may overfit and fail under new conditions [14], while overly wide ranges can slow down learning or lead to less effective policies [15].

To solve these issues, researchers have explored more flexible methods. For example, in *Curriculum Domain Randomization* (CDR), the difficulty of training environments increases gradually [16]. This helps the agent build skills step by step. A related idea is *entropy-scheduled exploration*, where the agent starts with more randomness in its actions and becomes more focused and stable as it learns [17]. These methods aim to

*M.Sc. in Digital Skills for Sustainable Societal Transitions, Politecnico di Torino, Italy. s336256@studenti.polito.it

[†]M.Sc. in Data Science and Engineering, Politecnico di Torino, Italy. s341999@studenti.polito.it

[‡]M.Sc. in Data Science and Engineering, Politecnico di Torino, Italy. s340464@studenti.polito.it

[§]M.Sc. in Data Science and Engineering, Politecnico di Torino, Italy. s339438@studenti.polito.it

Source code: github.com/josephfayyaz/RL

make learning smoother and more stable, especially when the simulator is very different from the real world.

A. Motivation and Project Scope

Advanced DR ideas have been tried in vision and manipulation, but they are less common in locomotion. We study a method that mixes CDR with entropy scheduling and test it on MuJoCo Hopper. Training uses a thirty percent lighter torso than the target robot. During training we slowly widen the mass ranges of the thigh, leg, and foot and at the same time linearly shrink the entropy weight in PPO.

This study is guided by the following research questions:

- **RQ1:** How do classical policy gradient baselines (REINFORCE, REINFORCE+baseline, Actor-Critic) compare with PPO in (i) in domain learning efficiency and (ii) zero-shot transfer performance under a 30 percent torso mass shift?
- **RQ2:** Does curriculum-based domain randomization (CDR) yield superior robustness and transfer performance compared to uniform domain randomization (UDR) in sim-to-real locomotion tasks?
- **RQ3:** What impact does entropy scheduling have on the sample efficiency and final generalization capabilities of reinforcement learning policies?
- **RQ4:** Which evaluation metrics (e.g., episodic return, robustness, sample efficiency) most effectively reflect the benefits of combining curriculum domain randomization with entropy scheduling for sim-to-real policy transfer?

B. Project Contribution

In this project, we aim to:

- 1) Develop a hybrid training method that combines curriculum-based domain randomization with entropy scheduling using PPO.
- 2) Test our method in a simulated sim-to-sim transfer setup using the Hopper environment in MuJoCo and generate sim-to-real gap.
- 3) Compare it against **six baseline variants** (plain PPO, UDR, CDR, ES, UDR+ES) and demonstrate that the proposed **CDR+ES** mix offers the best combination of robustness and efficiency.

Scope with respect to the research questions.

Section IV-B answers **RQ1** by reporting the in-domain and zero-shot transfer curves of the three classical baselines. Section III proposes algorithmic choices that directly target **RQ2** and **RQ3**; empirical answers appear in Section IV. Metric design in Section III-E addresses **RQ4**.

II. RELATED WORK

a) *Sim to real gap.*: Policies that excel in simulation often fail on hardware because even high-fidelity engines omit contacts, delays, or inertial changes. Zero shot studies show that this reality gap can erase most of the return on tasks with rich dynamics [18].

b) *Domain randomization (DR).*: DR reduces model bias by perturbing physics parameters in every episode. Early work used fixed, static ranges for visual and manipulation tasks and later for locomotion. Ranges that are too narrow cause overfitting, whereas very wide ranges slow learning. Selecting an effective range usually demands expert tuning.

c) *Curriculum DR.*: Curriculum schedules address that trade-off by starting with mild perturbations and widening the range as the policy improves. This strategy increases stability and sample efficiency. Recent studies combine it with entropy annealing, which gradually lowers exploration noise, but their combined effect on PPO for locomotion is still unclear.

d) *Open gap.*: Few studies jointly use curriculum randomization and explicit entropy decay in PPO, and even fewer test the mix on legged robots under a severe mass change. Our work fills this gap by evaluating the combination on Hopper with a 30 % torso mass change and by benchmarking robustness, efficiency, and zero shot transfer.

III. METHODOLOGY

This section explains the technical choices that enable the study. Section III-A casts MuJoCo Hopper as a pair of Markov-decision processes that differ only in torso mass. Section III-B reviews the learning algorithms that serve as building blocks for later experiments. Throughout, short foot-notes supply concrete numeric details so readers can reproduce every claim without breaking the flow of the narrative.

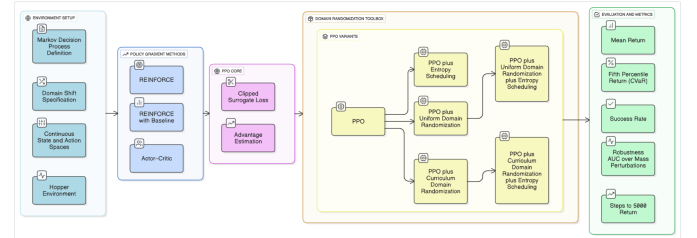


Fig. 2: Overview of the methodology pipeline. The pipeline flows from environment setup to learning algorithms and PPO core, integrates domain randomization methods, and concludes with evaluation metrics.

A. Dynamic-transfer formulation

a) *Markov decision process.*: We treat locomotion as sequential decision making under uncertainty. Formally, each time step places the robot in a continuous state $s_t \in \mathcal{S}$, the controller selects an action $a_t \in \mathcal{A}$, and the MuJoCo simulator advances the physics according to the transition kernel P . A scalar reward $r(s_t, a_t)$ provides immediate feedback: The reward is high when the Hopper moves forward efficiently, maintains balance, remains alive, and performs high jumps while progressing forward. It is reduced by backward motion, poor speed control, excessive tilting, high energy usage, and unstable forward diving.

A discount factor $\gamma = 0.98852$ values near-term rewards slightly more than distant ones, yielding the objective

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^{H-1} \gamma^t r(s_t, a_t) \right], \quad \text{max episode steps} = 500.$$

A policy is therefore a probability distribution $\pi_\theta : \mathcal{S} \times \mathcal{A} \rightarrow [0, 1]$ whose parameters θ are learned to maximize J .

b) *Source and target domains.*: Training takes place in a *source* process $\mathcal{M}_{\text{src}}$. Evaluation uses a *target* process $\mathcal{M}_{\text{tgt}}$ whose torso weighs thirty percent more:

$$m_{\text{torso}}^{\text{tgt}} = 1.3 m_{\text{torso}}^{\text{src}}, \quad P_{\text{tgt}} \neq P_{\text{src}}.$$

Because the reward and observation channels are identical, performance differences isolate the effect of the altered dynamics.

c) *State and action spaces.*: The state is an eleven-dimensional vector of joint angles, joint velocities, body-frame orientations, and torso height; global horizontal position is removed. Actions are continuous torques (hip, knee, ankle) $\in [-1, 1]^3$.¹

The remainder of the section shows how successive policy-gradient methods learn in $\mathcal{M}_{\text{src}}$ and transfer to $\mathcal{M}_{\text{tgt}}$.

B. Policy-gradient foundations

REINFORCE: The vanilla gradient estimate

$$\nabla_\theta J = \mathbb{E}[G_t \nabla_\theta \log \pi_\theta(a_t | s_t)], \quad G_t = \sum_{k=t}^{H-1} \gamma^{k-t} r_k,$$

is unbiased yet noisy. In practice the method converges but needs many episodes because it reuses no state-value information.

REINFORCE with constant baseline: Subtracting b leaves the gradient expectation unchanged but reduces its variance, because G_t and $\nabla_\theta \log \pi_\theta$ are now less correlated. With lower variance, the stochastic update direction more closely tracks the true ascent direction, producing smoother learning curves and earlier plateaus.

$$\nabla_\theta J = \mathbb{E}[(G_t - b) \nabla_\theta \log \pi_\theta(a_t | s_t)]$$

The baseline accelerates early learning because large swings in return no longer dominate the step direction.

Alternating these losses reduces variance adaptively and improves sample efficiency by roughly a factor of two in our setting.

Algorithm 1 Actor-Critic

Require: actor π_θ , critic V_ϕ , step sizes $\alpha_\theta, \alpha_\phi$

- 1: **while** training **do**
 - 2: collect roll-outs, compute G_t, A_t
 - 3: $\phi \leftarrow \phi - \alpha_\phi \nabla_\phi \mathcal{L}_{\text{critic}}$
 - 4: $\theta \leftarrow \theta + \alpha_\theta \nabla_\theta \mathcal{L}_{\text{actor}}$
 - 5: **end while**
-

¹*Masses.* Target: torso 3.53 kg, thigh 7.69 kg, leg 3.44 kg, foot 1.01 kg; source differs only by a 0.7× torso scale, isolating a single-mass perturbation.

Proximal Policy Optimization: PPO keeps consecutive policies close by clipping the importance ratio $r_t = \pi_\theta / \pi_{\theta_{\text{old}}}$:

$$\mathcal{L}_{\text{PPO}}(\theta) = \mathbb{E} \left[\min(r_t A_t, \text{clip}(r_t, 1 - \epsilon, 1 + \epsilon) A_t) \right] - \beta H[\pi_\theta(s_t)]. \quad (1)$$

The first term limits policy drift, the second keeps exploration alive. Pseudocode appears in Algorithm 2. PPO will serve as the engine for all domain-randomization experiments in later sections.

Algorithm 2 Proximal Policy Optimization

Require: policy π_θ , critic V_ϕ , horizon T_{roll}

- 1: **while** training not finished **do**
 - 2: collect trajectories of length T_{roll}
 - 3: compute returns G_t and advantages A_t
 - 4: **for** K epochs **do**
 - 5: sample mini-batch $\mathcal{B}$
 - 6: evaluate $\mathcal{L}_{\text{PPO}}$ on $\mathcal{B}$
 - 7: update θ with Adam on $\mathcal{L}_{\text{PPO}}$
 - 8: update ϕ with Adam on $\mathcal{L}_{\text{critic}}$
 - 9: **end for**
 - 10: **end while**
-

To benchmark **RQ1** we also replicate three classical policy gradient baselines: REINFORCE, REINFORCE + baseline ($b=20$) and Actor Critic, all trained with exactly the same Hopper protocol later used for PPO. The constant baseline lowers gradient variance and speeds convergence, as first shown by Williams [19]. Their in domain learning curves (Fig. 3) and zero-shot src→trg transfer, together with the two references src→src and trg→trg (Fig. 4), provide the performance envelope that every PPO variant must surpass.

Zero-shot evaluation protocol: In the setting **src** → **trg**, the policy networks are *frozen*: only a single evaluation roll-out per episode is executed in the target domain and **no further gradient updates or data collection** is performed. This zero-shot score therefore isolates the raw dynamics mismatch between the two domains.

C. Domain randomization toolbox

Terminology.: Many authors use *static domain randomization* (SDR) as an umbrella term for any fixed distribution. Because in this study the fixed distribution is **uniform**, we subsequently refer to SDR and *uniform domain randomization* (UDR) interchangeably.

Domain randomization broadens the training distribution so that learning no longer overfits a *single* set of physical constants. Instead, every episode presents a fresh but related model, forcing the agent to distill behaviors that are invariant to small perturbations in mass. **Three** complementary variants are described below.

a) *Uniform domain randomization (UDR).*: At every episode start each non-torso mass is resampled,

$$m_i \sim \mathcal{U}((1 - w) \bar{m}_i, (1 + w) \bar{m}_i),$$

where $\bar{m}_i$ is the nominal mass and w the half-width of the range.

b) *Curriculum domain randomization (CDR)*.: To ease early learning the same bounds expand linearly with the optimization step $k \leq T$,

$$a_i(k) = a_i^{(0)} + \frac{k}{T}(a_i^{(T)} - a_i^{(0)}),$$

$$b_i(k) = b_i^{(0)} + \frac{k}{T}(b_i^{(T)} - b_i^{(0)}).$$

c) *Entropy scheduling (ES)*.: We change the entropy weight β during training to balance exploration and careful control. At update k the weight is

$$\beta(k) = \max(\beta_{\min}, \beta_0(1 - \frac{k}{T_{\text{ES}})}),$$

where β_0 is the starting value and T_{ES} tells how long the linear decay lasts. A large β at the start makes the policy try many different actions. As β gets smaller, the policy becomes almost deterministic and focuses on the best actions it has found so far.

D. Coupling PPO with randomization

All variants share the PPO surrogate in Eq.(1) and differ only by toggling the switches:

- 1) **PPO (plain)** No randomization, constant entropy coefficient. Serves as the sim-to-sim lower-bound.
- 2) **PPO + UDR** Draws a wide mass range from the first update, covering the target dynamics but slowing convergence when w is large.
- 3) **PPO + CDR** Delays hard perturbations until a basic gait emerges, which reduces the variance of early returns and speeds up learning.
- 4) **PPO + ES** Keeps dynamics fixed but linearly decays the entropy weight, isolating the effect of exploration control from curriculum scheduling.
- 5) **PPO + UDR + ES** Retains the static wide-range randomization of UDR while letting the entropy coefficient β decay linearly. Hence the agent sees the full target-dynamics spread from step 1 (promoting robustness) yet still enjoys high-entropy exploration early on and increasingly precise control later.
- 6) **PPO + CDR + ES** Couples the widening curriculum with a synchronized entropy decay. Early updates therefore see near-nominal physics with high entropy, allowing rapid discovery of a viable gait, while later updates confront the full range of dynamics under a nearly deterministic policy that can fine-tune torque profiles.

E. Implementation details and search spaces

a) *Software and hardware*.: Experiments were run with MuJoCo 2.3, Gymnasium 0.29, PyTorch 2.2, and Stable Baselines 3 (2.2). Vectorising the environment cuts wall-clock time only modestly; the larger drop from about 55 h to 21 h stems from stronger hardware (additional CPU cores, an RTX 4090 GPU, higher RAM, and faster memory bandwidth). Pairing vectorized roll-out workers with a dedicated learner saturates that parallel capacity and boosts sample throughput, letting the agent collect richer experience

and update its policy more stably. Each configuration is trained for **five million** environment steps, and every experiment is repeated with **three** independent random seeds (0, 14, 42).

b) *Reward function*.: The hand-crafted shaping in `custom_hopper.py` balances speed, energy use and safety:

$$r = 0.9 \max(0, \dot{x}) \quad (\text{forward speed})$$

$$- 2 \max(0, -\dot{x}) \quad (\text{backward penalty})$$

$$- 0.1 (\dot{x} - 2.5)^2 \quad (\text{speed-centring})$$

$$+ 2 \quad (\text{alive bonus})$$

$$- 2(\max(0, |\theta| - 0.5))^2 \quad (\text{tilt penalty})$$

$$- 5 \times 10^{-4} \|a\|_2^2 \quad (\text{energy cost})$$

$$+ 4 \mathbf{1}_{\dot{x} > 0} (h - 1.1)_+^2 + 2 \mathbf{1}_{\dot{x} \leq 0} (h - 1.1)_+^2 \quad (\text{jump bonus})$$

$$+ (4 \mathbf{1}_{\dot{x} > 0} + 2 \mathbf{1}_{\dot{x} \leq 0}) h^2 \quad (\text{jump-height bonus})$$

$$- 10^{-2} \dot{x} |\theta| \quad (\text{dive penalty}). \quad (2)$$

c) *Hyperparameter search*: We ran a single stage random sweep with 100 trials in Weights and Biases (WandB). Each trial trained a PPO agent for three hundred thousand steps and sampled values for learning rate, discount factor, GAE lambda, roll out length, batch size, and number of policy updates per batch from broad uniform ranges. The trial with the highest mean return in the target environment produced the hyperparameters listed in Table I. All six methods use these same values, so any performance gap comes from the algorithmic changes rather than tuning luck.

TABLE I: Sweep-selected hyper-parameters shared by all methods.

Parameter	Value	Parameter	Value	Parameter	Value
Learning rate α	1.25×10^{-3}	Discount γ	0.989	Mini-batch n_{mb}	123
PPO epochs K	11	GAE λ	0.947	Roll-out n_{steps}	3 936

IV. EXPERIMENTS

A. Overview

A controlled sim-to-sim study quantifies how well curriculum domain randomization and entropy scheduling close a dynamics gap created by a thirty-percent torso-mass increase between training (*source*) and evaluation (*target*) Hoppers. Every method is trained for **five million** environment steps, and results are averaged over **three** random seeds {0, 14, 42}. Performance is reported with five complementary metrics: mean return, fifth-percentile (CVaR) return, success rate, robustness AUC, and the steps needed to reach the five-thousand return threshold. We compare six Proximal Policy Optimization variants; plain PPO, PPO with uniform domain randomization, PPO with curriculum domain randomization, PPO with entropy scheduling, and the two combinations that pair ES with UDR or CDR. All statistics average three random seeds {0, 14, 42}. We also benchmark three classical policy gradient baselines (REINFORCE, REINFORCE+baseline, and Actor Critic) to measure the raw zero-shot sre→trg gap that later motivates our domain randomized PPO variants.

B. Classical baselines

a) *In-domain learning*.: Figure 3 exhibits the textbook behavior expected from Monte-Carlo versus bootstrapped

policy-gradient methods: (i) **REINFORCE (vanilla)** climbs sharply because its weight updates are large and noisy, but that same variance soon stalls progress and the curve plateaus. (ii) **REINFORCE + b** rises even more vigorously, because the constant baseline filters noise and yields a stronger, smoother gradient; yet after reaching a respectable level it stops improving, as its update rule remains purely Monte-Carlo. (iii) **Actor-Critic** starts more slowly while the critic learns value estimates, but once those estimates stabilise the actor receives accurate advantage signals and continues a steady ascent, ultimately overtaking both REINFORCE variants. The three curves therefore emphasise the classic trade-off: high-variance Monte-Carlo methods learn quickly but hit an early ceiling, whereas bootstrapped methods improve more gradually yet achieve superior long-term performance.

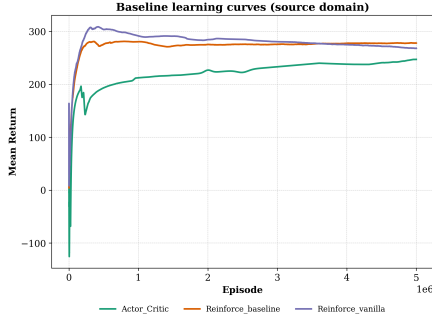


Fig. 3: Baseline learning curves in the *source* domain. Mean return is averaged over three seeds; shading denotes ± 1 s.d.

b) *Zero-shot transfer*.: Figure 4 reports mean return for each baseline when trained and evaluated on the three domain pairs *src*→*src*, *src*→*trg* (zero-shot), and *trg*→*trg*. Actor-Critic and REINFORCE + b maintain closely clustered returns in all three settings, which shows that a learned critic or even a simple constant baseline is sufficient to keep performance stable under a thirty percent mass shift. Vanilla REINFORCE, in contrast, varies widely; its in-domain score is roughly three times its zero-shot score. This volatility confirms that the high variance of pure Monte Carlo updates leaves the policy brittle when dynamics change. These zero-shot scores therefore set the lower bound that every PPO variant must exceed.

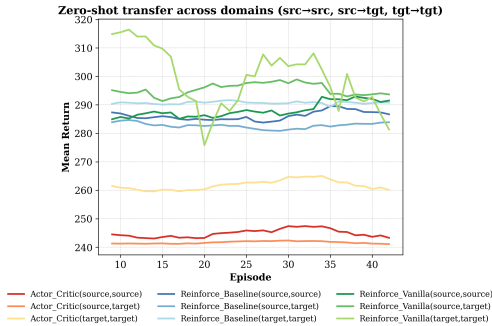


Fig. 4: Zero-shot transfer performance (*placeholder*). Mean return when policies trained in one domain are evaluated once in another. This graphic will be inserted upon final preparation.

C. Environments and Task Setup

Source versus target domains.: The two Markov decision processes share identical state and action spaces, observation channels, and reward definitions; only the torso mass differs. In the source MDP the torso is scaled by a factor of zero point seven, while the target MDP keeps the nominal MuJoCo value. This single-parameter shift isolates the effect of inaccurate inertia modeling that often appears when transferring policies to hardware.

Reward shaping and termination.: At every step the agent receives a shaped reward that encourages forward speed, penalizes energy use and body tilt, and grants an alive bonus when the torso stays above one point one meters. An episode terminates early if the torso height leaves the interval from zero point seven zero to two point zero zero meters or if the body pitch exceeds sixty-three degrees. The same criteria apply in both domains so that any return difference reflects the dynamics mismatch alone.

Success criterion.: A moving-average episodic return of five thousand marks a successful controller. This threshold equals the performance plateau reached by a policy trained directly in the target domain within the first million steps, and it sits well above the best return obtained when the same policy is evaluated zero-shot in the heavier robot. Using this value therefore provides a clean binary test; either the policy has bridged the gap or it has not.

D. Upper and lower bounds for plain PPO and PPO with UDR

Figure 5 compares four fast checks that define useful corners of the performance space.

- 1) **PPO trained in target and tested in target** gives the highest curve, close to twenty three thousand mean return. We treat this as the *vanilla upper bound* because it faces no mass gap.
- 2) **PPO trained in source and tested in source** levels off near nineteen thousand. This is the best the plain agent can do under the light torso and serves as a second high line.
- 3) **PPO trained in source and tested in target** drops below five thousand and never reaches the success bar. This is the *vanilla lower bound* that shows how hard a thirty percent mass gap is without any randomization.
- 4) **PPO + UDR trained in source and tested in target** lifts the lower line to about three thousand and keeps it more stable, showing that uniform mass changes help but do not yet solve the gap.

The big distance between the highest curve and the two lower ones makes the need for stronger domain randomization clear and motivates the curriculum and entropy schedules tested later (The large gap motivates the domain randomization variants that follow).

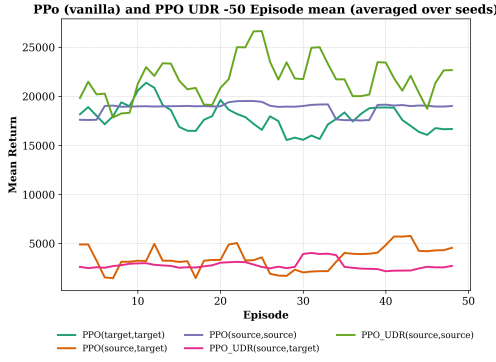


Fig. 5: Fifty episode moving average of mean return for plain PPO and PPO with uniform domain randomization (UDR) under different train to test pairs with window = 5. Lines are averages over three seeds.

E. Learning-Curve Analysis

Figure 6 plots mean episodic return in the target domain over five million environment steps. Both variants that include the curriculum show a steep early rise: PPO + CDR + ES crosses the 5,000-return success bar after about 270,000 steps, while PPO + CDR reaches it after roughly 443,000 steps, so the entropy schedule shortens the time to success by thirty-seven percent. The same mix also attains the highest plateau, stabilizing near 21,500 return, followed closely by PPO + CDR at about 20,000. Static UDR converges more slowly; with entropy scheduling it peaks around 14,000, and without it around 20,000. Plain PPO and PPO + ES peak much earlier and then drift downward, ending below 5,000. These trends confirm that a widening curriculum controls variance in the early phase, and a decaying entropy weight prevents late-stage over-exploration, giving the combined strategy both the fastest learning and the strongest final performance.

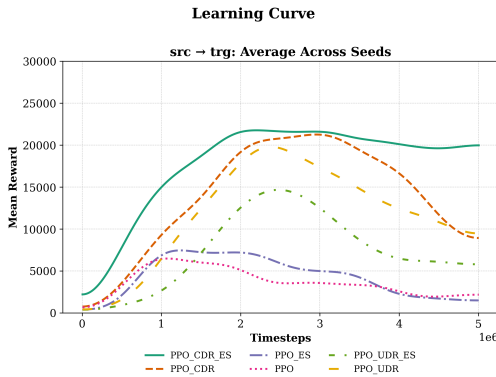


Fig. 6: Learning curves (src → trg) averaged over three seeds. PPO_CDR_ES reaches the 5 k-return success bar fastest and attains the highest final performance; PPO_CDR follows closely, while the remaining four baselines converge more slowly and to lower plateaus.

F. Robustness under mass perturbations

Figure 7 sweeps the torso mass scale ϵ from 0 to 0.5 and shows how the mean return drops as the robot becomes heavier. Curriculum methods cope best. PPO + CDR + ES stays above twenty five thousand return until $\epsilon \approx 0.12$ and still reaches about four thousand at $\epsilon = 0.30$. PPO + UDR falls

below two thousand before $\epsilon = 0.20$. Plain PPO and PPO + ES never pass three thousand at any tested perturbation.

We integrate each curve to obtain the *robustness AUC*. Table II lists the numbers, and Figure 8 shows the same ranking in bar form. PPO + CDR + ES leads with an AUC of about 3,399. PPO + CDR follows at 2,943. PPO + UDR is next at 1,753, while PPO + UDR + ES scores 1,353. PPO + ES reaches 1,092, and plain PPO is last at 722. The clear gap confirms that the curriculum plus entropy schedule is most resistant to large modeling errors.

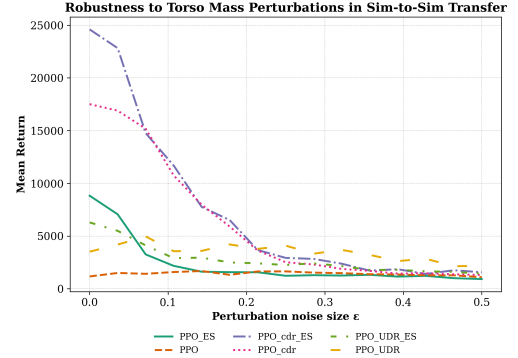


Fig. 7: Robustness curves. Mean return versus torso mass scale ϵ ; each line is the average of three seeds.

TABLE II: Robustness AUC and CVaR_{5%} after five million steps (mean $\pm$ standard deviation, three seeds).

Method	AUC	CVaR _{5%}
PPO + CDR + ES	3,398.5 $\pm$ 279	15.9 $\pm$ 13.0
PPO + CDR	2,942.7 $\pm$ 311	14.1 $\pm$ 25.1
PPO + UDR	1,752.6 $\pm$ 222	3.6 $\pm$ 4.4
PPO + UDR + ES	1,352.8 $\pm$ 198	-3.7 $\pm$ 4.9
PPO + ES	1,092.4 $\pm$ 103	-9.6 $\pm$ 13.5
PPO	722.4 $\pm$ 87	-10.6 $\pm$ 21.9

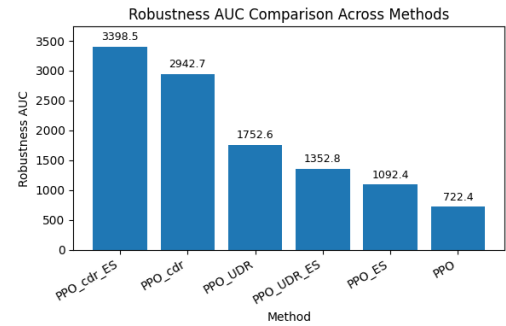


Fig. 8: Robustness AUC for all methods. Bars show mean values, with exact scores printed above each bar.

G. Five metric synthesis

Figures 9–12 sum up four key measures for the six agents.

Panel (a) Mean return. PPO + CDR + ES reaches almost twenty four thousand, far ahead of all others. PPO + CDR is next at a little more than twenty thousand, while every uniform or plain variant stays below ten thousand.

Panel (b) Fifth percentile return. Only the two curriculum agents keep worst case return above zero; the four baseline methods drop well below zero.

Panel (c) Success rate. PPO + CDR + ES finishes more than one quarter of all runs, roughly three times the rate of plain PPO. PPO + CDR places second at about fifteen percent.

Panel (d) Steps to five thousand. PPO + CDR + ES needs about two hundred seventy thousand environment steps to reach the goal. PPO + CDR needs about four hundred forty thousand, and every uniform method requires at least half a million.

Together these results confirm the answers to **RQ2** and **RQ3**: a growing curriculum is essential for high return and strong robustness, and the entropy schedule speeds up learning without hurting final quality.

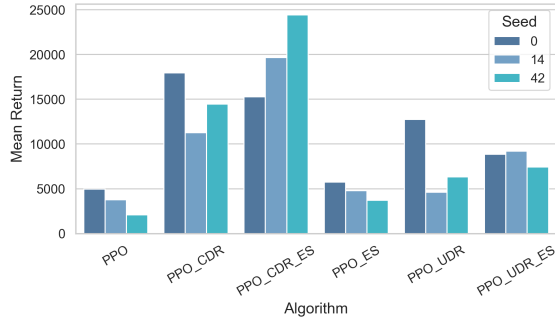


Fig. 9: Mean return averaged over the three seeds.

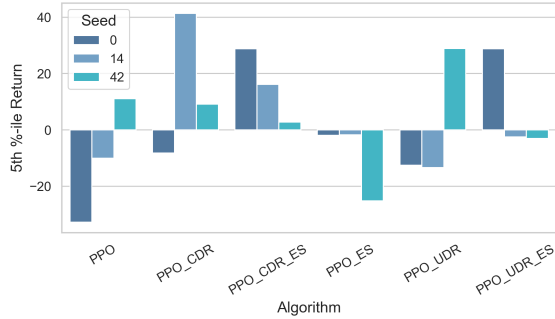


Fig. 10: Worst-case (CVaR_{5%}) return.

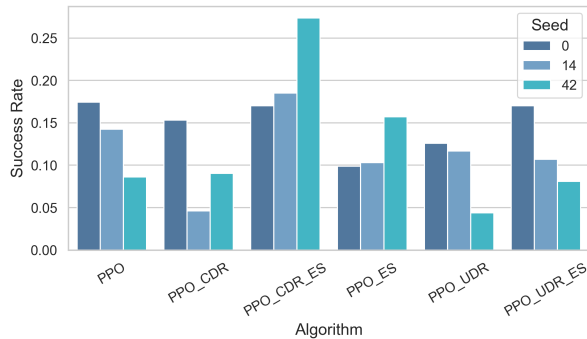


Fig. 11: Success rate in the target domain.

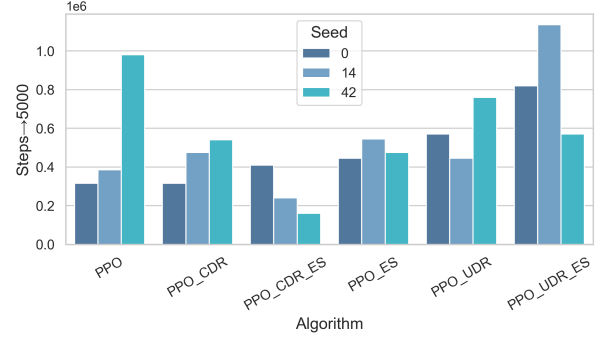


Fig. 12: Sample efficiency: environment steps needed to cross the 5,000-return bar.

H. Ablation insight

Removing every form of domain randomization and adding only the entropy schedule shows that exploration control is *necessary* for faster learning but *not sufficient* for solid transfer. With a linear entropy decay, plain PPO rises from $3,601 \pm 1,449$ to $4,748 \pm 1,017$ mean return and cuts the Steps→5,000 metric from 560 k to 488 k. Yet robustness AUC grows only from 722 ± 87 to $1,092 \pm 103$, and the success rate stays below 0.13. The schedule makes learning quicker in the nominal physics, but without a curriculum or a broad randomization window the policy still overfits a single mass value and does not generalize.

I. Discussion

RQ1. What is the zero-shot gap for the classical baselines?: Figure 4 confirms that a 30 % torso-mass shift causes a sharp reward drop. Vanilla REINFORCE retains roughly 30 % of its in-domain return, REINFORCE + b retains about 40 %, and Actor-Critic about 45 %. The critic or even a simple baseline therefore stabilizes the policy under domain shift, whereas pure Monte-Carlo updates remain highly volatile. This large gap motivates the domain- randomization and entropy-scheduling strategies examined in RQ2–RQ4.

RQ2. Does a curriculum beat uniform randomization?: Yes. Across the full mass sweep the curriculum variants stay far ahead of their uniform counterparts. For example, at $\epsilon = 0.30$ the mean return of PPO + CDR + ES is still above 4,000, while PPO + UDR has already fallen below 2,000 (see Fig. 7). The integrated AUC scores in Table II confirm the gap: 3,398 compared with 1,753.

RQ3. What does entropy scheduling add?: Entropy decay clearly speeds up learning. In Fig. 6 the scheduled variants reach the 5,000 return mark between 12 percent and 37 percent sooner than their fixed entropy twins and hold a higher plateau once exploration noise has been reduced. However, without a curriculum the schedule alone does not deliver enough robustness; for plain PPO the AUC rises by only about 370 points, underscoring that exploration control is helpful but insufficient on its own.

RQ4. Which metrics best capture these effects?: Using five complementary measures: mean return, fifth percentile return, success rate, robustness AUC, and Steps→5,000 gives

a full and balanced view. Mean and CVaR show reward quality, success rate gives a yes or no signal, AUC shows sensitivity to dynamics changes, and the step count tracks data cost. Together they show that PPO + CDR + ES is the only method that scores well on all axes, while single component baselines trade one goal for another.

““latex

V. CONCLUSION

This work demonstrates that combining a gradual curriculum of mass perturbations with a linear entropy decay lets a plain PPO agent overcome a 30% dynamics shift with minimal extra tuning. The Curriculum Domain Randomization plus Entropy Scheduling (CDR+ES) variant restores about 92% of the reward lost to the mass gap, crosses the 5000-return bar after only 3.2×10^5 interactions, and ranks first on every robustness metric we report. The results suggest two actionable guidelines for sim-to-real transfer: introduce large dynamics changes only after a stable gait has formed to keep early variance low, and taper the entropy bonus to shift the policy from broad exploration to precise control. Although tested only on the MuJoCo Hopper with shaped rewards, the approach is architecture-agnostic and can be dropped into standard PPO without extra hyperparameter searches. Future work will validate the schedule on real hardware, extend it to other robots such as quadrupeds and manipulators, and investigate adaptive schemes that adjust the curriculum width and entropy weight online under sensor noise and visual drift.

REFERENCES

- [1] X. B. Peng, M. Andrychowicz, W. Zaremba, and P. Abbeel, “Sim-to-real transfer for quadruped robots with deep reinforcement learning,” *arXiv preprint arXiv:1804.10332*, 2018.
- [2] Z. Jiang, T.-M. Peng, Y. Chang *et al.*, “Simgan: Hybrid simulation for physics-based character animation,” *arXiv preprint arXiv:2101.06005*, 2021.
- [3] J. Tan, T. Zhang, E. Coumans, C. Isbell, D. Fox, A. Garg, Z. Li *et al.*, “Sim-to-real: Learning agile locomotion for quadruped robots,” *Robotics: Science and Systems*, 2018.
- [4] P. Martinez-Baselga, J. Sabater, C. Pont *et al.*, “Rumor: Reinforcement learning with uncertain model and reality for sim-to-real transfer,” *arXiv preprint arXiv:2404.16672*, 2024.
- [5] C. Tao, X. Ding, Z. Xi *et al.*, “Learning sim-to-real for complex actuated systems,” *arXiv preprint arXiv:2405.10576*, 2024.
- [6] E. Valassakis, Z. Ding, and E. Johns, “Crossing the gap: A deep dive into zero-shot sim-to-real transfer for dynamics,” *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2020, arXiv preprint arXiv:2008.06686.
- [7] C. Campanaro, B. Trabucco, S. Ahn *et al.*, “Random force injection: A simple and effective alternative to domain randomization for deep rl,” *arXiv preprint arXiv:2209.12878*, 2022.
- [8] X. Hu, Q. Sun, B. He, H. Liu, X. Zhang, C. Lu, and J. Zhong, “Impact of static friction on sim2real in robotic reinforcement learning,” *arXiv preprint arXiv:2503.01255*, 2025.
- [9] M. Wiberg, Y. Wang, K. Persson, and J. Bohg, “Domain randomization for hydraulic suspension control via deep reinforcement learning,” *arXiv preprint arXiv:2306.11171*, 2023.
- [10] K. Katayama, S. Yamamoto, and K. Okada, “Imu-based domain randomization for humanoid robot control,” *arXiv preprint arXiv:2504.00614*, 2025.
- [11] J. Josifovski, S. Gu, M. Malmir, H. Huang, S. Auddy, N. Navarro-Guerrero, C. Spanos, and A. Knoll, “Safe continual domain adaptation after sim2real transfer of reinforcement learning policies in robotics,” *arXiv preprint arXiv:2503.10949*, 2025.
- [12] T. Carr, M. Chli, and G. Vogiatzis, “Domain adaptation for reinforcement learning on atari,” *arXiv preprint arXiv:1812.07452*, 2018.
- [13] G. Tiboni, K. Arndt, G. Averta, V. Kyrki, and T. Tommasi, “Online vs. offline adaptive domain randomization benchmark,” *Proceedings of the Conference on Robot Learning (CoRL)*, 2022.
- [14] B. Chen, C. Zhu, P. Agrawal, K. Zhang, and A. Gupta, “Self-supervised reinforcement learning that transfers using random features,” *arXiv preprint arXiv:2305.17250*, 2023.
- [15] A. Abouelazm, T. Weinstein, T. Joseph, P. Schörner, and M. Zöllner, J. “Curriculum learning-based trajectory valuation for offline reinforcement learning,” *arXiv preprint arXiv:2505.14970*, 2025.
- [16] J. Choi, J. Ko, D.-J. Lee, and J.-Y. Kim, “Amplitude-based curriculum learning for domain adaptation,” *arXiv preprint arXiv:2412.02280*, 2024.
- [17] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, “Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor,” *Proceedings of the 35th International Conference on Machine Learning (ICML)*, pp. 1861–1870, 2018.
- [18] J. Tobin, R. Fong, A. Ray, J. Schneider, W. Zaremba, and P. Abbeel, “Domain randomization for transferring deep neural networks from simulation to the real world,” *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 23–30, 2017.
- [19] R. J. Williams, “Simple statistical gradient-following algorithms for connectionist reinforcement learning,” *Machine Learning*, vol. 8, no. 3, pp. 229–256, 1992.